SolarPredict

Complete Final Year Project Defense Guide

Solar Energy Forecasting and Microgrid Optimization System for Douala and Maroua, Cameroon

Project Area	Description
Machine Learning	XGBoost Model II predicts hourly global horizontal irradiance (GHI).
Backend	FastAPI exposes prediction, dashboard, history, microgrid, alert, and reporting APIs.
Frontend	React + Vite interface with dashboard, prediction, history, microgrid, reports, and about pages.
Database	SQLite stores generated prediction records for analytics and reports.
Deployment	Vercel configuration for the frontend with API routing support.

Use this document as a defense preparation guide. It explains the project problem, architecture, folder structure, main code files, features, API endpoints, model pipeline, test plan, and common oral defense questions.

1. Project Overview

SolarPredict is a full-stack decision-support system for forecasting solar irradiance and helping microgrid operators decide how to use solar panels, batteries, and grid support. The system focuses on two Cameroonian cities with different climate profiles: Douala and Maroua.

- Douala represents a coastal, humid, high-rainfall environment.
- Maroua represents a hotter, drier Far North environment.
- The target variable is ALLSKY_SFC_SW_DWN, interpreted in the app as solar irradiance/GHI in W/m2.
- The operational output is not only a prediction number; the app converts predictions into recommendations, battery schedules, alerts, and seasonal reports.

Problem Statement

Solar energy is variable because it depends on weather, time of day, season, location, and atmospheric conditions. For a microgrid, this variability affects how much energy can be produced, when batteries should be charged, and when the grid or load shedding may be needed. SolarPredict addresses this by forecasting solar irradiance and using the forecast to support energy-management decisions.

Main Objectives

1. Collect and prepare historical solar/weather data for Douala and Maroua.
2. Train and compare machine-learning models for solar irradiance forecasting.
3. Select the best model and integrate it into a backend API.
4. Fetch live weather forecasts and transform them into the same features used during training.
5. Expose prediction results through a React dashboard.
6. Simulate microgrid battery, load, import, and export behavior.
7. Provide alerts and seasonal reporting to support operational decisions.

Expected Users

- Students and researchers studying renewable-energy forecasting.
- Microgrid operators who need a simple decision-support prototype.
- Energy planners comparing solar behavior between Cameroonian climate zones.
- Supervisors or examiners evaluating a complete data-science software project.

2. System Architecture

The application follows a clear layered architecture: the frontend collects user inputs and displays charts, the backend coordinates weather retrieval and prediction, the model layer loads saved XGBoost models, and SQLite stores prediction history for analytics.

```
React/Vite Frontend
  -> api/client.js
  -> FastAPI Backend (/api/*)
  -> Open-Meteo hourly weather forecast
  -> Feature engineering with Model II columns
  -> City-specific XGBoost .pkl model
  -> SQLite prediction history
  -> Dashboard, microgrid simulation, alerts, and reports
```

End-to-End Prediction Flow

1. The user selects a city and forecast horizon in the UI.
2. The frontend calls the FastAPI endpoint, usually POST /api/predict or GET /api/dashboard.

3. The backend validates the city against the CITIES configuration.
4. Open-Meteo forecast data is fetched for temperature, humidity, wind speed, precipitation, and shortwave radiation.
5. features.py converts date/time/weather values into the 21 Model II features.
6. model_loader.py loads the correct city-specific XGBoost model and runs inference.
7. Negative values are clamped to zero because solar irradiance cannot be physically negative.
8. Predictions are saved in SQLite and returned with weather snapshots and microgrid advice.
9. The frontend renders charts, tables, cards, alerts, and energy-balance results.

3. Complete Folder Structure

Path	Purpose
backend/	Python FastAPI backend. Contains API routes, services, database access, weather client, model loading, and microgrid logic.
backend/app/main.py	Application entry point. Defines all REST endpoints and coordinates prediction, dashboard, reporting, and microgrid workflows.
backend/app/config.py	Central settings: project paths, city coordinates, model filenames, Model II feature order, battery limits, alert thresholds, and defaults.
backend/app/database.py	SQLite schema creation/migration plus functions to save and read prediction records.
backend/app/features.py	Feature engineering logic that mirrors the training notebooks.
backend/app/weather.py	Open-Meteo async HTTP client with short cache for forecast data.
backend/app/model_loader.py	Loads .pkl models with joblib and runs predictions.
backend/app/microgrid.py	Simulates hourly solar generation, battery SoC, grid import/export, and self-sufficiency.
backend/app/schemas.py	Pydantic request and response models used for validation and API documentation.
backend/app/services/	Separated services for alerts, analytics, reports, and microgrid recommendation rules.
frontend/	React + Vite frontend application.
frontend/src/App.jsx	Defines client-side routes.
frontend/src/api/client.js	Central API wrapper used by all React pages.
frontend/src/components/	Reusable UI components: layout, cards, recommendation badge, battery gauge, alert banner.
frontend/src/pages/	Main screens: Dashboard, Prediction, History, Microgrid, Reports, About.
frontend/src/utils/	Shared localStorage utility for microgrid settings.
Model/	Saved trained models used by the production backend.
Data/Raw/	Original datasets for Douala and Maroua.
Data/Processed/	Cleaned and feature-engineered datasets used for modeling.
training/	Training notebooks and CSV comparison results.
docs/	Project documentation PDFs and this generated guide.
*.ipynb	Notebook stages for preprocessing, EDA, feature engineering, model training, and validation.

4. Data and Machine Learning Pipeline

Raw and Processed Data

The raw datasets are stored in Data/Raw. The processed datasets are stored in Data/Processed. Model II uses weather variables plus engineered time and season features. The target variable is ALLSKY_SFC_SW_DWN.

Feature Group	Fields	Reason
Calendar	YEAR, MO, DY, HR	Solar radiation follows strong hour, month, and season patterns.
Weather	T2M, RH2M, PRECTOTCORR, WS10M	Temperature, humidity, rain, and wind help explain atmospheric conditions.
Cyclic time	hour_sin, hour_cos, month_sin, month_cos, day_sin, day_cos	Cyclic encoding lets the model understand that 23:00 is close to 00:00 and December is close to January.
Derived flags	day_of_year, is_daytime, is_rainy	Adds physical and calendar meaning to the raw weather data.
Transforms	rain_log1p, ws10m_log1p	Reduces skew in rain and wind variables while safely handling zero values.
Season flags	season_dry, season_rainy	Represents local climate differences between Douala and Maroua.

Why Model II Is Important

Model II avoids solar leakage. Leakage happens when the model is given features that are too directly related to the target, such as another measured solar radiation value from the same timestamp. A model with leakage can look very accurate during testing but fail in real operation. Model II uses weather and time variables that are realistically available at prediction time.

Selected Model Results

City	Best Model	MAE	RMSE	R2
Douala	XGBoost	24.83	53.32	0.9503
Maroua	XGBoost	26.55	60.20	0.9657

MAE and RMSE measure prediction error in W/m2. R2 measures how much variance is explained by the model. The high R2 values indicate that XGBoost captures the relationship between time, weather, and irradiance well for both cities.

Why XGBoost Was Chosen

- It handles non-linear relationships between weather, time, and irradiance.
- It performs well on tabular data.
- It is faster and easier to deploy than deep learning for this dataset size.
- It outperformed Random Forest and LSTM in the saved Model II comparison summary.
- The saved .pkl files are easy to load in the FastAPI backend using joblib.

5. Backend Code Explanation

backend/app/main.py

main.py is the API controller layer. It defines the FastAPI app, enables CORS for the React frontend, initializes the database and models on startup, closes the HTTP client on shutdown, and exposes all endpoints. It does not contain all business logic directly; instead it delegates to database.py, weather.py, features.py, model_loader.py, microgrid.py, and service modules.

Endpoint	Purpose
GET /	Simple message for users who open the backend in a browser.
GET /api/health	Confirms the API is running and returns supported cities.
GET /api/cities	Returns Douala and Maroua metadata with coordinates.
GET /api/dashboard	Enhanced dashboard payload with live weather, 24 h forecast, microgrid schedule, alerts, metrics, and seasonal comparison.
GET /api/dashboard/basic	Legacy simple dashboard response.
POST /api/predict	Runs an hourly prediction workflow and saves results in SQLite.
GET /api/history	Returns saved prediction records, optionally filtered by city.
GET /api/microgrid/recommendation	Tests the simple irradiance-to-advice rule.
POST /api/microgrid/optimize	Runs a battery/grid simulation from predicted solar generation.
GET /api/reports/seasonal	Returns dry vs rainy seasonal performance summary.
GET /api/reports/seasonal/csv	Exports seasonal prediction records as CSV.

backend/app/config.py

config.py is the single source of truth for paths, supported cities, model filenames, features, target name, battery limits, alert thresholds, and default microgrid settings. This makes the project easier to maintain because city metadata and feature order are not repeated across files.

```
CITIES = {
  "douala": {"latitude": 4.0511, "longitude": 9.7679, "model_file": "douala_xgb_model_ii.pkl"},
  "maroua": {"latitude": 10.5956, "longitude": 14.3247, "model_file": "maroua_xgb_model_ii.pkl"},
}

MODEL_II_FEATURES = [
  "YEAR", "MO", "DY", "HR", "T2M", "RH2M", "PRECTOTCORR", "WS10M",
  "hour_sin", "hour_cos", "month_sin", "month_cos", "day_sin", "day_cos",
  "day_of_year", "is_daytime", "is_rainy", "rain_log1p", "ws10m_log1p",
  "season_dry", "season_rainy"
]
```

backend/app/features.py

features.py transforms raw forecast weather into the exact columns expected by the trained XGBoost model. The most important defense point is that the same feature engineering used during training must be repeated during live inference. If the feature order or formulas change, the model may receive wrong inputs.

- _season_flags(month, city) defines dry/rainy season logic for each city.
- build_model_ii_row(...) creates one feature row from datetime and weather variables.
- build_feature_dataframe(...) converts rows into a pandas DataFrame in the exact MODEL_II_FEATURES order.

backend/app/weather.py

weather.py is responsible for external forecast data. It uses httpx.AsyncClient to call the Open-Meteo forecast API. It requests temperature_2m, relative_humidity_2m, wind_speed_10m, precipitation, and shortwave_radiation. It also caches each city/hour request for five minutes so repeated dashboard refreshes are faster.

backend/app/model_loader.py

model_loader.py loads each city-specific XGBoost model from the Model folder. Models are cached in memory so they are not reloaded on every request. The predict function selects columns in the expected order and clamps negative model outputs to zero.

backend/app/database.py

database.py manages SQLite. On startup, init_db creates or migrates the predictions table. The database stores city, weather variables, prediction value, and timestamp. This history supports the History page, dashboard statistics,

model metrics, and seasonal reports.

Column	Meaning
id	Primary key.
city	douala or maroua.
temperature	Temperature in degrees Celsius.
humidity	Relative humidity percentage.
wind_speed	Wind speed in m/s.
precipitation	Precipitation in mm.
prediction	Predicted irradiance in W/m2.
timestamp	Forecast timestamp.

backend/app/microgrid.py

microgrid.py converts predicted irradiance into estimated solar energy and simulates battery/grid behavior. It respects a 20 percent minimum state of charge and a 95 percent maximum state of charge. When solar generation is greater than load, the battery charges first and any remaining surplus is exported. When solar generation is lower than load, the battery discharges down to the minimum SoC and any remaining deficit is imported from the grid.

```
solar_kwh = irradiance_wm2 * panel_area_m2 * panel_efficiency / 1000
net = solar_kwh - hourly_load

if net >= 0:
    charge battery until 95 percent SoC, export surplus
else:
    discharge battery down to 20 percent SoC, import remaining deficit
```

backend/app/schemas.py

schemas.py defines Pydantic models for request validation and structured responses. This is why invalid values such as zero battery capacity or more than 168 prediction hours are rejected before they reach the business logic.

Service Modules

File	Explanation
microgrid_recommendation.py	Maps irradiance to high, moderate, or low solar energy advice.
alert_service.py	FR-7 alert system. Triggers a critical alert when next 6 h solar energy is below 0.5 kWh/m2 and battery SoC is below 40 percent.
analytics_service.py	Builds operational model metrics and dry/rainy seasonal comparison from stored prediction history.
report_service.py	Builds seasonal JSON reports and CSV export content.

6. Frontend Code Explanation

React Application Structure

The frontend is a React single-page application built with Vite. App.jsx defines routes, Layout.jsx provides the sidebar and shared shell, api/client.js centralizes backend communication, and each page implements one major feature.

File	Role
App.jsx	Defines routes for Dashboard, Prediction, History, Microgrid, Reports, and About.
api/client.js	Wraps fetch calls, normalizes backend errors, and exposes functions such as api.predict and api.dashboard.
components/Layout.jsx	Sidebar navigation and page outlet.
components/StatCard.jsx	Reusable metric card for weather, GHI, and energy totals.
components/RecommendationBadge.jsx	Colored recommendation display based on high/moderate/low level.
components/BatteryGauge.jsx	Visual battery state-of-charge component with min/max markers.
components/AlertBanner.jsx	Displays normal, warning, or critical alert status.
utils/microgridSettings.js	Stores microgrid settings in localStorage and syncs Dashboard with Microgrid page.

Dashboard Page

Dashboard.jsx is the main operational screen. It fetches GET /api/dashboard with selected city and microgrid settings. It shows current weather, current GHI, recommendation status, 24-hour GHI chart, battery SoC gauge, seasonal comparison, model metrics, alert banner, and a 24-hour microgrid schedule.

Prediction Page

Prediction.jsx lets the user run forecasts for a selected city and number of hours. It calls POST /api/predict, displays the first-hour recommendation, renders a line chart, and shows a table with weather and microgrid status for forecast rows.

History Page

History.jsx reads from GET /api/history. It supports filtering by city and displays saved SQLite records. This demonstrates persistence: predictions remain available after they are generated.

Microgrid Page

Microgrid.jsx lets users configure city, battery capacity, daily load, and panel area. It calls POST /api/microgrid/optimize and displays predicted solar, grid import, grid export, self-sufficiency, state-of-charge, energy-balance chart, and schedule table.

Reports Page

Reports.jsx implements FR-9. It loads dry vs rainy seasonal comparison, lets the user download CSV data, and exports the chart as PNG by converting the rendered SVG chart to a canvas image.

7. Functionalities and Uses

Functionality	Use
City selection	Compare Douala and Maroua solar behavior.
Live weather integration	Uses current forecast conditions instead of fixed sample inputs.
Hourly irradiance forecast	Predicts solar availability for planning.
Prediction history	Stores operational records for tracking and analytics.
Microgrid recommendation	Converts prediction into practical high/moderate/low energy advice.

Functionality	Use
Battery/grid simulation	Shows how much energy comes from solar, battery, or grid.
SoC limits	Protects battery by keeping charge between 20 percent and 95 percent.
Alert system	Warns when low solar and low battery occur together.
Seasonal reports	Compares dry and rainy season performance.
CSV/PNG export	Supports documentation and reporting for project defense or analysis.

8. Testing Plan

A strong defense should include both technical tests and demonstration tests. The table below explains what to test, how to test it, and what result is expected.

Test	How	Expected Result
Backend health	Open /api/health	Returns status ok and city list.
Invalid city validation	Call /api/dashboard?city=yaounde	HTTP 400 unsupported city.
Prediction flow	POST /api/predict with city=douala, hours=24	Returns 24 prediction rows and saves them.
Feature order	Inspect MODEL_II_FEATURES and build_feature_dataframe	DataFrame columns match training order.
Model files	Start backend	Models load without FileNotFoundError.
History persistence	Run prediction then GET /api/history	New records appear in SQLite history.
Recommendation thresholds	Test 399, 400, 700, 701 W/m2	Low, moderate, moderate, high respectively.
Microgrid limits	Run optimize with 24 h horizon	Battery SoC never goes below 20 percent or above 95 percent.
Alert rule	Use low 6 h predictions and low SoC in unit test	Critical alert with load-shedding priority.
CSV export	Open /api/reports/seasonal/csv	Returns text/csv content.
Frontend build	Run npm run build	Vite creates production build without errors.
UI integration	Use every page in browser	No blank charts, errors, or broken navigation.

Suggested Unit Tests to Add

- Test _season_flags for Douala and Maroua months.
- Test build_model_ii_row returns all 21 required features.
- Test get_microgrid_recommendation threshold boundaries.
- Test evaluate_alert for no alert, warning, and critical states.
- Test simulate_microgrid respects battery SoC min/max.
- Test database init creates the predictions table.
- Test API validation rejects invalid cities and invalid microgrid parameters.

Manual Defense Demonstration Script

1. Start backend: cd backend; python -m uvicorn app.main:app --reload --port 8000.
2. Start frontend: cd frontend; npm run dev.
3. Open the app and show the Dashboard for Douala.
4. Switch to Maroua and explain why climate differences matter.
5. Run a 24-hour prediction and explain the chart/table.
6. Open History to show saved records.

7. Open Microgrid, change battery/load/panel values, and explain self-sufficiency.
8. Open Reports, show dry vs rainy comparison, and download CSV.
9. Open /docs in FastAPI to show automatic API documentation.

9. Defense Explanation by Topic

How to Explain the Feature Engineering

I converted raw weather and time data into a richer set of features. The model receives both direct weather variables and derived variables. Cyclic features represent periodic behavior, daytime flags represent physical solar availability, log transforms handle skewed rain and wind values, and season flags represent local climate patterns.

How to Explain the Microgrid Optimization

The microgrid algorithm is a rule-based energy-balance simulation. It uses predicted irradiance to estimate hourly solar generation. If generation is higher than load, the battery charges until the maximum SoC, then excess energy is exported. If generation is lower than load, the battery discharges down to the minimum SoC, then the grid supplies the remaining demand.

How to Explain Alerts

The alert rule combines forecast risk and storage risk. Low solar alone is a warning, and low battery alone is also a warning. A critical alert occurs only when the next 6-hour solar energy is below 0.5 kWh/m2 and battery SoC is below 40 percent. That avoids unnecessary alarm when one condition is bad but the other is still safe.

Limitations

- Forecast quality depends on the external weather provider.
- The microgrid model is simplified and does not include inverter losses, battery degradation, tariffs, or dynamic load profiles.
- Operational model metrics are currently computed from stored predictions, not from future measured ground truth.
- Only two cities are supported because trained models and city-specific climate logic are defined for Douala and Maroua.
- The app requires the saved model files to be present in the Model folder.

Future Improvements

- Add real measured solar observations for live accuracy tracking.
- Add authentication and user-specific microgrid profiles.
- Support more Cameroonian cities with trained city models.
- Add advanced optimization using electricity tariffs, battery degradation, and load priority classes.
- Deploy backend and frontend with production monitoring.
- Add automated tests in pytest and frontend integration tests.

10. Common Oral Defense Questions

Question	Strong Answer
Why did you choose XGBoost?	Because it performs strongly on tabular data, captures non-linear weather/time relationships, trains faster than deep learning, and achieved the best Model II results for both cities.
What is solar leakage?	It is when training features contain information too close to the target, causing unrealistic accuracy. Model II avoids leakage by using weather and time features available at prediction time.
Why separate models by city?	Douala and Maroua have different climate patterns. Separate models allow each city to learn its own relationship between weather, season, and irradiance.
Why use cyclic encoding?	Hour and month are circular. 23:00 and 00:00 are close, and December and January are close. Sine/cosine encoding helps the model understand that.

Question	Strong Answer
Why clamp negative predictions?	Solar irradiance cannot be negative. Clamping protects the application from physically impossible model outputs.
What does SQLite do?	It stores generated prediction records for history, dashboard statistics, analytics, and seasonal reports.
How does the microgrid simulation work?	It compares hourly solar generation with load, charges/discharges the battery within SoC limits, and imports/exports the remaining energy.
What is the alert condition?	Critical alert occurs when the next 6 hours have less than 0.5 kWh/m2 solar energy and battery SoC is below 40 percent.
How would you validate the deployed system?	Compare predictions with measured irradiance, monitor MAE/RMSE/R2 over time, and test API/UI workflows regularly.
What is the main contribution?	A complete end-to-end system: data preparation, ML forecasting, live API integration, database history, decision-support UI, microgrid simulation, alerts, and reports.

11. Quick Revision Checklist

- Know the problem: solar variability affects microgrid planning.
- Know the data: Douala and Maroua, target ALLSKY_SFC_SW_DWN, weather/time features.
- Know the model: XGBoost Model II, no solar leakage, saved .pkl files.
- Know the backend: FastAPI endpoints, feature engineering, model loading, SQLite.
- Know the frontend: Dashboard, Prediction, History, Microgrid, Reports, About.
- Know the microgrid rules: high > 700, moderate 400 to 700, low < 400 W/m2.
- Know the battery limits: 20 percent minimum, 95 percent maximum, 50 percent default.
- Know FR-7: alert condition uses 6-hour solar and battery SoC.
- Know FR-8: enhanced dashboard includes forecast, SoC gauge, alerts, metrics, seasonal comparison.
- Know FR-9: seasonal report and CSV export.
- Prepare a live demo and a fallback screenshot/PDF in case the network is slow.

12. Run Commands

```
# Backend
cd backend
pip install -r requirements.txt
python -m uvicorn app.main:app --reload --port 8000

# Frontend
cd frontend
npm install
npm run dev

# Backend API docs
http://127.0.0.1:8000/docs

# Frontend app
http://localhost:5173
```

13. Final Defense Summary

SolarPredict demonstrates a complete final-year engineering project because it moves from data preparation and model training to real application deployment. It predicts solar irradiance for two Cameroonian cities, converts predictions into operational recommendations, stores results for analysis, simulates microgrid behavior, warns about risky low-energy conditions, and produces seasonal reports. The system is explainable, modular, and practical enough to defend as both a machine-learning project and a software engineering project.